On Combiners for Verifiable Random Functions

Basil Rohner (brohner@ethz.ch)
Bachelor's Thesis, ETH Zürich
May 6, 2026

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …
- A broken assumption breaks every construction that relies on it

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …
- A broken assumption breaks every construction that relies on it

Don’t bet on a single assumption.
Combine candidates based on different ones.

Research Question

Can we *combine* many verifiable random function (VRF) instances such that the resulting VRF is secure if *enough* candidates are secure?

Research Question

Can we *combine* verifiable random function (VRF) instances such that the resulting VRF is secure if *k*-out-of-*n* candidates are secure?

Research Question

Can we *combine* verifiable random function (VRF) instances in a black-box fashion such that the resulting VRF is secure if *k*-out-of-*n* candidates are secure?

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

A VF is a purely syntactic object, we define security properties on top of it.

Properties of Verifiable Functions

ε-Correctness

Honestly generated image-proof pairs verify with probability at least ε:

$$\Pr\left[\begin{array}{l}(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda}) \\ \quad x \leftarrow_{\$} \{0,1\}^{\lambda} \quad : \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk},x,y,\pi) = 1 \\ (y,\pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk},x)\end{array}\right] \geq \varepsilon(\lambda)$$

VF is *correct* if it is 1-correct and *statistically correct* if it is $(1-\varepsilon)$-correct and $\varepsilon \in \mathsf{negl}(\lambda)$.

Properties of Verifiable Functions

Unique Provability

For every public key pk and every point x, at most one image y is provable:

$$\mathrm{Vfy}_{\mathrm{VRF}}(\mathsf{pk}, x, y_0, \pi_0) = \mathrm{Vfy}_{\mathrm{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1.$$

Properties of Verifiable Functions

(ε, t)-Pseudorandomness

For any adversary $\mathcal{A}$ running in time at most t:

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Pseudorandomness experiment

Experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathsf{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Properties of Verifiable Functions

(ε, t)-Unpredictability

For any adversary $\mathcal{A}$ running in time at most t, playing the prf experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$:
The advantage $\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ is bounded from above by

$$\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Unpredictability experiment

Experiment $\mathbf{Exp}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathrm{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathrm{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathrm{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x)$
3 : **return** y

Properties of Verifiable Functions

(ε, t)-Weak Pseudorandomness

For any adversary $\mathcal{A}$ running in time at most t, playing the prf experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$, the advantage $\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ is bounded from above by:

$$\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

A VF is *pseudorandom* if it is (ε, t)-pseudorandom for $\varepsilon \in \mathsf{negl}(\lambda)$ and $t \in \mathsf{poly}(\lambda)$.

Weak Pseudorandomness experiment

Experiment $\mathbf{Exp}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathrm{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathrm{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathrm{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x)$
3 : **return** y

Verifiable Random Functions

Verifiable Random Function

A *verifiable random function* (VRF) is a VF that is:

- Correct
- Uniquely Provable
- Pseudorandom

Verifiable Unpredictable Functions

Verifiable Unpredictable Function

A *verifiable unpredictable function* (VUF) is a VF that is:

- Correct
- Uniquely Provable
- Unpredictable

Weak Verifiable Unpredictable Functions

Weak Verifiable Unpredictable Function

A *weak verifiable unpredictable function* (WVUF) is a VF that is:

- Correct
- Uniquely Provable
- Weakly Pseudorandom

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]
- Resettable Zero-Knowledge proofs [MR01]

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]
- Resettable Zero-Knowledge proofs [MR01]
- Distributed Leader Election in PoS blockchains [KRDO17; DGKR18; BASV23]

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

Robust $(n; k)$ black-box P-combiner Π_P satisfy:

- Give oracle access to n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$
- Implements primitive P
- Is secure if at least k candidates are secure

With inefficient we mean adversaries with access to a PSPACE complete oracle.

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

Robust $(n; k)$ black-box P-combiner Π_P satisfy:

- Give oracle access to n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$
- Implements primitive P
- Is secure if at least k candidates are secure **even against inefficient adversaries**

With inefficient we mean adversaries with access to a PSPACE complete oracle.

Example

Algorithm $\Pi_{\mathsf{PRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{PRF}}(k_1, k_2, x)$

1 : $y_1 \leftarrow \mathcal{O}_1(k_1, x)$
2 : $y_2 \leftarrow \mathcal{O}_2(k_2, x)$
3 : **return** $y_1 \oplus y_2$

Figure: Robust $(2; 1)$ Black-Box PRF-Combiner.

Robust Black-Box DAG Combiners

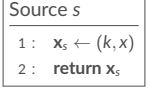
Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

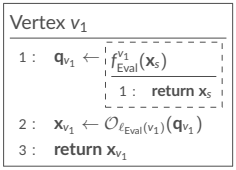
Robust $(n; k)$ black-box DAG P-combiner $\widetilde{\Pi}_{\mathsf{P}}$ consist for every algorithm:

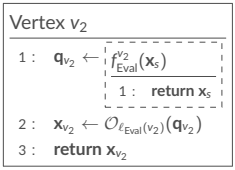
- Directed acyclic graph $\mathcal{G}$
- Source $s \in V(\mathcal{G})$
- Sink $t \in V(\mathcal{G})$
- Vertex labeling $\ell : V(\mathcal{G}) \to \mathsf{P} \times [n]$
- PPT Turing machines $\{f^v\}_{v \in V(\mathcal{G}) \setminus \{s\}}$

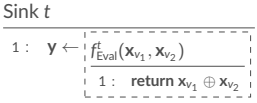
Execute $\widetilde{\Pi}_{\mathsf{P}}$ in topological order. At $v \in V(\mathcal{G})$ evaluate f^v on all predecessor outputs and query $\mathcal{O}_{\ell(v)}$.

; [-latex] (x) -> (v1); [-latex] (v1) -> (t); [-latex] (x) -> (v2); [-latex] (v2) -> (t);


Source s
1 : $\mathbf{x}_s \leftarrow (k, x)$
2 : return $\mathbf{x}_s$


Vertex v_1
1 : $\mathbf{q}_{v_1} \leftarrow f^{v_1}_{\text{Eval}}(\mathbf{x}_s)$
1 : return $\mathbf{x}_s$
2 : $\mathbf{x}_{v_1} \leftarrow \mathcal{O}_{\ell_{\text{Eval}}(v_1)}(\mathbf{q}_{v_1})$
3 : return $\mathbf{x}_{v_1}$

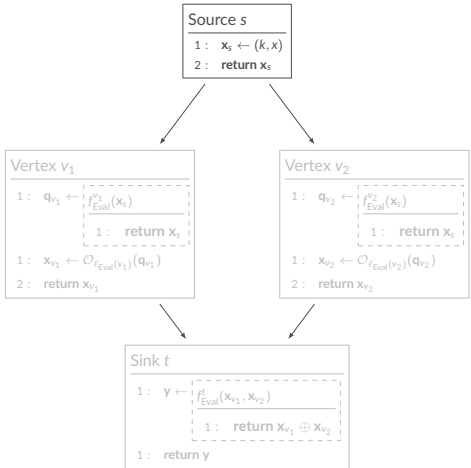

Vertex v_2
1 : $\mathbf{q}_{v_2} \leftarrow f^{v_2}_{\text{Eval}}(\mathbf{x}_s)$
1 : return $\mathbf{x}_s$
2 : $\mathbf{x}_{v_2} \leftarrow \mathcal{O}_{\ell_{\text{Eval}}(v_2)}(\mathbf{q}_{v_2})$
3 : return $\mathbf{x}_{v_2}$


Sink t
1 : $\mathbf{y} \leftarrow f^{t}_{\text{Eval}}(\mathbf{x}_{v_1}, \mathbf{x}_{v_2})$
1 : return $\mathbf{x}_{v_1} \oplus \mathbf{x}_{v_2}$
2 : return $\mathbf{y}$

eval(val)

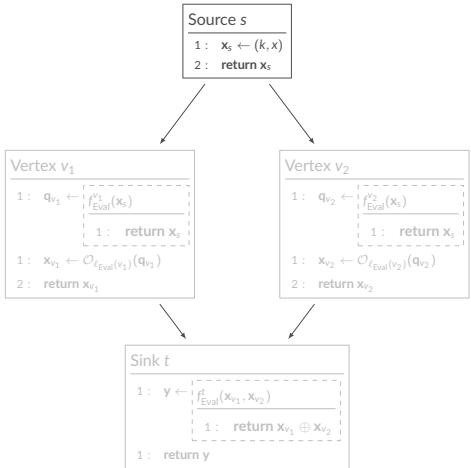
Example

- symbolic, simplifies reasoning
- produces the input



Example

- symbolic, simplifies reasoning
- produces the input



Equivalence

- Robust $(n; k)$ black-box DAG Combiner $\implies$ Robust $(n; k)$ black-box combiners

Equivalence

- Robust $(n; k)$ black-box DAG Combiner $\implies$ Robust $(n; k)$ black-box combiners
- Robust $(n; k)$ black-box combiners $\implies$ Robust $(n; k)$ black-box DAG combiner

Equivalence

- Robust $(n; k)$ black-box DAG Combiner $\implies$ Robust $(n; k)$ black-box combiners
- Robust $(n; k)$ black-box combiners $\implies$ Robust $(n; k)$ black-box DAG combiner

Sufficient to show (im)possibility of a robust $(n; k)$ black-box DAG combiner.

The 1-out-of-2 Combiner

- Simplest non-trivial case, only two candidates

The 1-out-of-2 Combiner

- Simplest non-trivial case, only two candidates
- Construction extends to robust 1-out-of-*n* black-box combiner

The 1-out-of-2 Combiner

- Simplest non-trivial case, only two candidates
- Construction extends to robust 1-out-of-n black-box combiner
- Impossibility extends to robust $\lfloor n/2 \rfloor$-out-of-n black-box combiner

What we show

There exists no robust $(2; 1)$ black-box combiner for VRF.

Setup

Suppose Π_{VRF} is a robust $(2;1)$ black-box DAG VRF-combiner. The proof works by

- Candidates $\mathcal{O}_1, \mathcal{O}_2$ of which at least one is secure
- Adversary $\mathcal{A}$ against
- $\mathcal{O}_{vrf}$: a secure VRF
- $\mathcal{O}_{uppc}$: a *trivial*, uniquely provable and perfectly correct VF:
  - $\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) = (\mathsf{sk}_{def}, \mathsf{pk}_{def})$
  - $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = (y_{def}, \pi_{def})$
  - $\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1 \iff y = y_{def}$
- $\mathcal{O}_{vrf}$ a secure VRF $\implies$ $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$ a secure VRF

Goal: Adversary $\mathcal{A}^{\mathsf{PSPACE}, \mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$ against psuedorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$.
Assume: $\mathcal{SK} = \{0,1\}^{s(\lambda)}$ for $s \in \mathsf{poly}(\lambda)$ for all candidates.

The Adversary

Assume we have a Turing machine (inefficient) $\mathsf{EvalSim}_{\mathsf{VRF}}$ satisfying:

- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \bot$ if $\mathsf{Eval}^{\gamma}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq \mathsf{Eval}^{\gamma}_{\mathsf{VRF}}(\mathsf{sk}, x)$
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes *no queries to any oracles*.
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*.

For $x \in \{0,1\}^{\lambda}$, $(y, \pi) = \Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}} \mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and $\tau = \mathsf{trace}(\Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi))$.

The Adversary

Adversary $\mathcal{A}_1^{\mathsf{PSPACE},\mathcal{O}_{vrf},\mathcal{O}_{uppc},\mathcal{O}_{\mathsf{eval}}}(\mathsf{pk})$:

1. Sample $x_i \leftarrow_{\$} \{0,1\}^\lambda$ for all $i \in [s(\lambda)+\lambda]$
2. For all $i \in [s(\lambda)+\lambda]$ query
$$(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$$
3. For all $i \in [s(\lambda)+\lambda]$ compute the verification trace
$$\tau_i \leftarrow \mathsf{trace}\left(\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x_i, y_i, \pi_i)\right)$$
4. Sample a candidate secret key sk_{cand} using the PSPACE-complete oracle
$$\mathsf{sk}_{cand} \leftarrow_{\$} \{\mathsf{sk} \in \mathcal{SK} : \forall i \in [s(\lambda)+\lambda], \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top\}$$

The Adversary

$$\mathbf{Adv}^{\mathsf{prf}}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}}(\lambda,\mathcal{A}) \geq \frac{1}{2} - \frac{1}{2^{\lambda+1}} - \frac{1}{|\mathcal{Y}|}.$$

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Given $(\mathsf{sk}_{cand}, x, y, \pi, \tau)$, $\mathsf{EvalSim}_{\mathsf{VRF}}$ is computed by:

- Follow the DAG $\mathcal{G}_{\mathsf{Eval}}$, describing $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$, in some topological ordering
- For every vertex v corresponding to a query to $\mathcal{O}_{uppc}$ answer with default value
- For every vertex v corresponding to a query to $\mathcal{O}_{vrf}$ check τ:
  - If $\mathbf{q} \in \tau$: answer accordingly
  - If $\mathbf{q} \notin \tau$: branch for every possible answer
- Return $\top$ iff all branches produce the image y

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Given $(\mathsf{sk}_{cand}, x, y, \pi, \tau)$, $\mathsf{EvalSim}_{\mathsf{VRF}}$ is computed by:

- Follow the DAG $\mathcal{G}_{\mathsf{Eval}}$, describing $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$, in some topological ordering
- For every vertex v corresponding to a query to $\mathcal{O}_{uppc}$ answer with default value
- For every vertex v corresponding to a query to $\mathcal{O}_{vrf}$ check τ:
  - If $\mathbf{q} \in \tau$: answer accordingly
  - If $\mathbf{q} \notin \tau$: branch for every possible answer
- Return $\top$ iff all branches produce the image y

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Assume we have a Turing machine (inefficient) $\mathsf{EvalSim}_{\mathsf{VRF}}$ satisfying:

- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \bot$ if $\mathsf{Eval}^{\gamma}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq \mathsf{Eval}^{\gamma}_{\mathsf{VRF}}(\mathsf{sk}, x)$
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes *no queries to any oracles*.
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*.

For $x \in \{0,1\}^{\lambda}$, $(y, \pi) = \Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and $\tau = \mathsf{trace}(\Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi))$.

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*:

- $\mathsf{EvalSim}_{\mathsf{VRF}}$ in NPSPACE since every branch is an execution of $\mathsf{Eval}_{\mathsf{VRF}}$
- NPSPACE $\subseteq$ PSPACE

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \perp$ if $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq y$:

- All queries made to $\mathcal{O}_{uppc}$ answered the same $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x)$
- All queries made to $\mathcal{O}_{vrf}$ with $\mathbf{q} \in \tau$ answered the same
- For all queries made to $\mathcal{O}_{vrf}$ with $\mathbf{q} \notin \tau$ there exists a branch which is the same

$\Longrightarrow$ At least one branch evaluates to $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq y$.

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$.

- Suppose $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \perp$
- Then there exist answers to queries to $\mathcal{O}_{vrf}$ such that the output is $y' \neq y$
- Modify $\mathcal{O}_{uppc}$ to make it a secure VRF such that $\mathcal{O}^*_{uppc}\big|_{\widetilde{\mathcal{SK}}} = \mathcal{O}_{uppc}\big|_{\widetilde{\mathcal{SK}}}$
- Modify $\mathcal{O}_{vrf}$ to produce the transcript of the branch, $\mathcal{O}^*_{vrf}$
- $\mathcal{O}_{vrf}$ and $\mathcal{O}^*_{vrf}$ only differ on queries $\mathbf{q} \notin \tau$ and indistinguishable for verification

Now we have:

$$y = \Pi_{\mathsf{VRF}}^{\mathcal{O}^*_{uppc}, \mathcal{O}_{vrf}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}, x) \neq \Pi_{\mathsf{VRF}}^{\mathcal{O}^*_{uppc}, \mathcal{O}^*_{vrf}}.\mathsf{Eval}^y_{\mathsf{VRF}}(\mathsf{sk}, x) = y^*$$

How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$.
Now we have:

$$y = \Pi_{\mathsf{VRF}}^{\mathcal{O}^*_{uppc}, \mathcal{O}_{vrf}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x) \neq \Pi_{\mathsf{VRF}}^{\mathcal{O}^*_{uppc}, \mathcal{O}^*_{vrf}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x) = y^*$$

Since the verification algorithms are however the same on queries $\mathbf{q} \in \tau$, the proof π verifies identically in both executions.

$\implies$ Two distinct values $y \neq y^*$ both verify under the same (pk, x, π), breaking **unique provability** of Π_{VRF}.

From $(2; 1)$ to $(n; \lfloor n/2 \rfloor)$

Given a robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner Π_{VRF}, build a robust $(2; 1)$ combiner $\widetilde{\Pi}_{\mathsf{VRF}}$:

- initialize the first $\lfloor n/2 \rfloor$ candidates of Π_{VRF} with $\mathcal{O}_1$
- initialize the remaining $\lceil n/2 \rceil$ candidates with $\mathcal{O}_2$

If either $\mathcal{O}_1$ or $\mathcal{O}_2$ is a secure VRF, at least $\lfloor n/2 \rfloor$ candidates of Π_{VRF} are secure, so $\widetilde{\Pi}_{\mathsf{VRF}}$ is a robust $(2; 1)$ combiner.

Contraposition with the main theorem yields the impossibility of robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiners.

Result

Theorem. Impossibility 1-out-of-2 combiner.

There is **no** robust $(2; 1)$ black-box VRF-combiner.

Corollary. Impossibility $\lfloor n/2 \rfloor$-out-of-n combiner.

There is **no** robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner.

Conjecture. Impossibility $(n-1)$-out-of-n combiner.

There is **no** robust $(n; n-1)$ black-box VRF-combiner.

What We Showed

Additionally the same proof shows that:

Lemma. Impossibility $\lfloor n/2 \rfloor$-out-of-n VUF-combiner.

There is **no** robust $(2; 1)$ black-box VRF-combiner.

Lemma. Impossibility $\lfloor n/2 \rfloor$-out-of-n wVRF-combiner.

There is **no** robust $(2; 1)$ black-box VRF-combiner.

Beyond Impossibility

Let assume *unique provability* for all *n* candidates:

- *Then construction similar to combiner for PRF is applicable*

Too strong of an assumption?

- No, unique provability is an information theoretic property and typically proven.

Robust 1-out-of-n Black-Box Combiner for UP-VRF

Adversary $\mathcal{A}$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathsf{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner
- Conjectured impossibility for robust $(n; n-1)$ black-box VRF-combiner

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner
- Conjectured impossibility for robust $(n; n-1)$ black-box VRF-combiner
- Constructed robust $(n; 1)$ black-box UP-VRF-combiner

References I

[BASV23] J. Burdges, H. K. Alper, A. Stewart, and S. Vasilyev. Sassafras and Semi-Anonymous Single Leader Election. Cryptology ePrint Archive, Report 2023/031, 2023.

[DGKR18] B. David, P. Gaži, A. Kiayias, and A. Russell. Ouroboros Praos: An Adaptively-Secure, Semi-synchronous Proof-of-Stake Blockchain. In *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 66–98. Springer, 2018.

[GO93] S. Goldwasser and R. Ostrovsky. Invariant Signatures and Non-Interactive Zero-Knowledge Proofs are Equivalent (Extended Abstract). In *CRYPTO'92*, volume 740 of *LNCS*, pages 228–245. Springer, 1993.

[KRDO17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 357–388. Springer, 2017.

[MR01] S. Micali and L. Reyzin. Soundness in the Public-Key Model. In *CRYPTO 2001*, volume 2139 of *LNCS*, pages 542–565. Springer, 2001.

Where the Proof breaks down

- In the 1-out-of-2 case we rely on the fact that for one candidate we can always guess the responses perfectly, and for the other the obscure queries are mutually irrelevant in the sense that under the correct secret key the image y stays the same. That means that for all tests on the correct secret keys sk^* the output remains consistent through all runs. For any $n > 2$ and $k = n - 1$ we have at least two candidate whose outputs we cannot predict perfectly. While mutual independence holds for